

Numerical solving of non-linear Schrödinger equation

March 14, 2011

1 Initial objective

On first stage, it was shown that OpenFOAM could be used in principle for solving non-linear Schrödinger equation. The equation to solve is

$$i\frac{\partial\Psi}{\partial t} + C_0\Delta\Psi + C_1|\Psi|^2\Psi = 0 \quad (1)$$

with Dirichlet or Neumann boundary conditions, and asymmetric initial conditions. The boundary is supposed to be shaped as a cylinder or as a sphere, for test purposes.

The software that was suggested to use is OpenFOAM. To learn how to use this software, I run several tutorial examples from

<http://www.openfoam.com/docs/user/tutorials.php>.

2 Mesh generation

Running OpenFOAM consists of (1) generating polyhedral mesh for use in Finite Volume Method, (2) executing a program, or “solver” that integrates the given differential equation, and (3) display of results.

Example of using of one of mesh-generating utilities is shown on Fig. 1. The radius of cylinder is $R = 1$ and the size $L = 5$. The cylinder is represented as a curvilinear hexagonal structure, with subdivision $32 \times 32 \times 32$ and uniform grading.

3 Heat transfer equation

One of closest analogues of non-linear Schrödinger equation available in OpenFOAM is the heat transfer, of diffusion equation

$$\frac{\partial\Psi}{\partial t} - D\Delta\Psi = 0 \quad (2)$$

where D is a coefficient of thermal conductivity.

```

FoamFile
{
  version      2.0;
  format       ascii;
  class        dictionary;
  object       blockMeshDict;
}
// *****

convertToMeters 1.0;

vertices
(
  (1 0 0)
  (0 -1 0)
  (-1 0 0)
  (0 1 0)
  (1 0 5)
  (0 -1 5)
  (-1 0 5)
  (0 1 5)
);

blocks
(
  hex (4 5 6 7 0 1 2 3) (32 32 32) simpleGrading (1 1 1)
);

edges
(
  arc 0 1 (0.707 -0.707 0)
  arc 1 2 (-0.707 -0.707 0)
  arc 2 3 (-0.707 0.707 0)
  arc 3 0 (0.707 0.707 0)
  arc 4 5 (0.707 -0.707 5)
  arc 5 6 (-0.707 -0.707 5)
  arc 6 7 (-0.707 0.707 5)
  arc 7 4 (0.707 0.707 5)
);

patches
(
  {
    wall bound
    (
      (0 1 2 3)
      (4 5 6 7)
      (0 1 5 4)
      (1 2 6 5)
      (2 3 7 6)
      (3 0 4 7)
    )
  }
);

mergePatchPairs
(
);
// *****

```

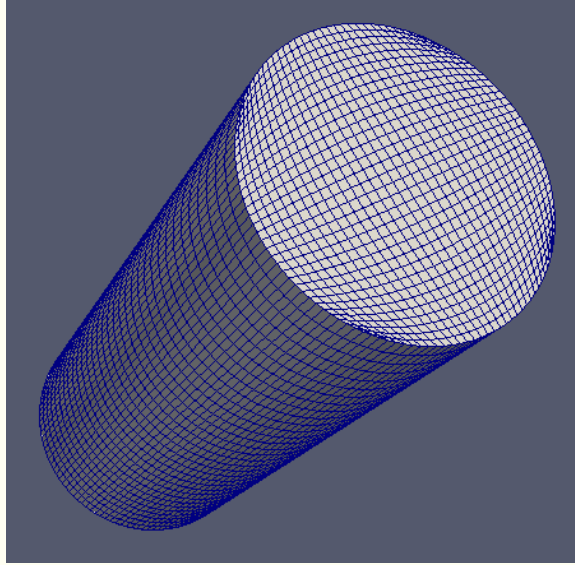


Figure 1. Example of `blockMeshDict` file and the mesh for a cylinder constructed with use of `blockMesh` utility.

The “solver” `laplacianFoam.C` of this equation is given by Listing 1. Notice that “`fvm::laplacian`” means that the Laplacian is calculated implicitly as a matrix, i.e. we deal here with implicit Euler method of integration.

Listing 1. Solving heat transfer equation

```

#include "fvCFD.H"
int main(int argc, char *argv[])
{
  # include "setRootCase.H"
  # include "createTime.H"
  # include "createMesh.H"
  # include "createFields.H"
  // *****
  Info<< "\nCalculating temperature distribution\n" << endl;
  while (runTime.loop())
  {
    Info<< "Time=_" << runTime.timeName() << nl << endl;
    # include "readSIMPLEControls.H"
    for (int nonOrth=0; nonOrth<=nNonOrthCorr; nonOrth++)
    {
      solve
      (
        fvm::ddt(T) - fvm::laplacian(DT, T)
      );
    }
  }
}

```

```
#      include "write.H"
      Info<< "ExecutionTime==" << runTime.elapsedCpuTime() << "s"
          << "ClockTime==" << runTime.elapsedClockTime() << "s"
          << nl << endl;
    }
    Info<< "End\n" << endl;
    return 0;
}
```

The time and write control input data are given by Listing 2. Notice that integration occurs between $t = 0$ and $t = 0.5$ with a time step $\Delta t = 0.0005$ and write time interval 0.002.

Listing 2. Time and write control data organized in `controlDict` file

```
application      laplacianFoam;
startFrom        startTime;
startTime        0;
stopAt           endTime;
endTime          0.5;
deltaT           0.0005;
writeControl      runTime;
writeInterval     0.002;
purgeWrite       0;
writeFormat       ascii;
writePrecision    6;
writeCompression uncompressed;
timeFormat        general;
timePrecision     6;
runTimeModifiable yes;
```

The input constant D is given by Listing 3, i.e. in this case $D = 0.1$. Numbers in square brackets specify dimensionality of the heat transfer constant, m^2/sec .

Listing 3. Transport properties organized in `transportProperties` file.

```
DT              DT [ 0 2 -1 0 0 0 0 ] 0.1;
```

Heat distribution at initial time $t = 0$ is defined as

$$T(x, y, z)_{(t=0)} = \exp(-(x - 0.5)^2 - y^2 - z^2).$$

This function was calculated by other program, and the results were entered into the file `T` in the subfolder 0 which is designated for storage of the initial data.

The results of calculations are shown in Fig. 2

4 Modification for non-linear Schrödinger equation

The “solver” for the heat transfer equation was modified in order to allow a non-zero imaginary part, which is denoted as `TI`, see Listing 4. Notice that “`fvc::laplacian`” means

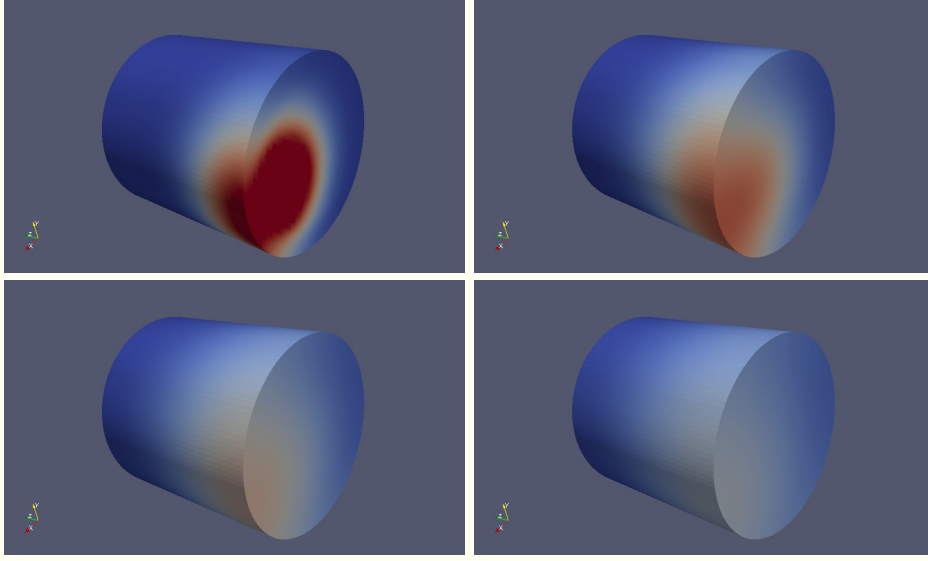


Figure 2. Results of calculations as visualized by `paraFoam` utility. Heat distribution at initial time $t = 0$ is shown at upper left. Subsequent panels show calculated heat distribution at $t = 0.08, 0.16$, and 0.24 .

that the Laplacian is calculated explicitly as a value, i.e. we deal here with *explicit* Euler method of integration. The reason of using the explicit method is inability of easily to adjust an implicit method for the case of two coupled equations (for real and imaginary parts).

Listing 4. Modification of `laplacianFoamM.C` solver by adding a variable `TI` for imaginary part of the function. The non-linearity is represented by the term $\pm CT * T * (T * T + TI * TI)$.

```
#include "fvCFD.H"
int main(int argc, char *argv[])
{
#   include "setRootCase.H"
#   include "createTime.H"
#   include "createMesh.H"
#   include "createFields.H"
    Info<< "\nCalculating temperature distribution\n" << endl;
    while (runTime.loop())
    {
        Info<< "Time=_" << runTime.timeName() << nl << endl;

#       include "readSIMPLEControls.H"

        for (int nonOrth=0; nonOrth<=nNonOrthCorr; nonOrth++)
        {
            solve
```

```

        (
            fvm::ddt(T) - DT*fvc::laplacian(TI) - CT*TI*(T*T+TI*TI)
        );
        solve
        (
            fvm::ddt(TI) + DT*fvc::laplacian(T) + CT*T*(T*T+TI*TI)
        );
    }
#    include "write.H"
    Info<< "ExecutionTime=" << runTime.elapsedCpuTime() << "s"
        << "ClockTime=" << runTime.elapsedClockTime() << "s"
        << nl << endl;
}
    Info<< "End\n" << endl;
    return 0;
}

```

5 Courant number

Explicit method suffer from numerical instability in case when Courant number $\Delta t/(\Delta x)^2$ (not sure of possible pre-factors) is larger than one, where Δx is grid spacing. Fig. 3 shows developing of instability that starts at corners of the curvilinear hexahedron, where the grid spacing becomes exceptionally small. Finally, the code gives "float overflow" numerical error. Parameters of equation (1) are $C_0 = 1$ and $C_1 = 0$. The cylinder has radius $R = 1$, length $L = 2$. The number of mesh subdivisions is 32 in each direction.

6 Collapsing and exploding solitons

6.1 Formulation of problem

Here, we solved non-linear Schrödinger equation with two non-linear terms, cubic and quintic. The equation to solve is

$$i\frac{\partial\Psi}{\partial t} + \Delta\Psi + C_1|\Psi|^2\Psi - C_2|\Psi|^4\Psi = 0, \quad (3)$$

with Dirichlet boundary conditions.

If the constants C_1 and C_2 are positive, then the equation describes self-guiding light beams organized in a solitonic structure [1, 2], with two traverse coordinates x, y and the third coordinate t in the direction of the beam propagation.

Here however we solve the equation in four coordinates, x, y, z , and t . The cubic non-linear term, $|\Psi|^2\Psi$, describes an effect of self-focusing, and the quintic term, $|\Psi|^4\Psi$, describes an effect of de-focusing. In order for a spatial soliton to become stable, the cubic and quintic non-linearities must have an opposite sign. We illustrate this behavior by numerical simulations with various parameters C_1 and C_2 .

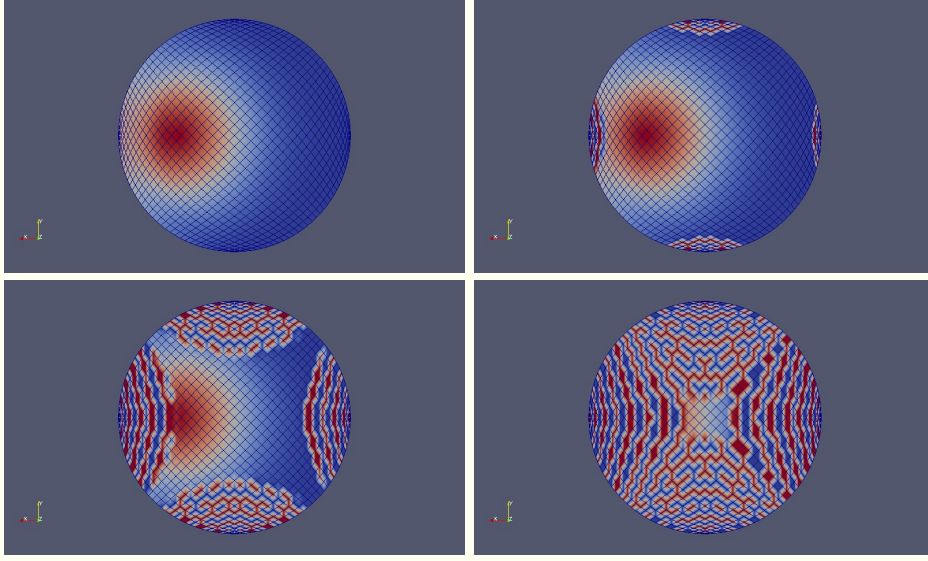


Figure 3. Divergence caused by Courant number larger than 1. The panels show calculated absolute value of the wave function of equation (1) at $t = 0, 0.0024, 0.0048$, and 0.0074 .

6.2 Initial conditions, mesh, and time step

The equation is solved inside a ball of radius 20 with the center at the origin. The mesh was obtained from a cubic grid $128 \times 128 \times 128$ by applying the OpenFOAM tool `snappyHexMesh`, see Figure 4.

The initial condition is given as

$$\Psi_{(t=0)} = A(x, y, z) e^{-\frac{(x-x_0)^2 + (y-y_0)^2 + (z-z_0)^2}{2R^2}}, \quad (4)$$

$$x_0 = 0.1, \quad y_0 = 0.2, \quad z_0 = 0, \quad R = 7, \quad (5)$$

and

$$A(x, y, z) = 1 + 0.1 \frac{x + iy}{(x^2 + y^2)^{1/2}} \quad (6)$$

where the second term is added in order to break rotation symmetry.

The time step is chosen as $\Delta t = 0.0025$ that is around 50% of maximal possible time step for which Courant number allows convergence of the method. The function was computed up to $t_{\max} = 15$. Execution times equals to 12 hours.

6.3 Linear Schrödinger equation

Here, equation (3) is solved for the case $C_1 = C_2 = 0$ (linear case). As it could be expected, the wave packet slowly disperses with passing of time, and remains Gaussian, see Figure

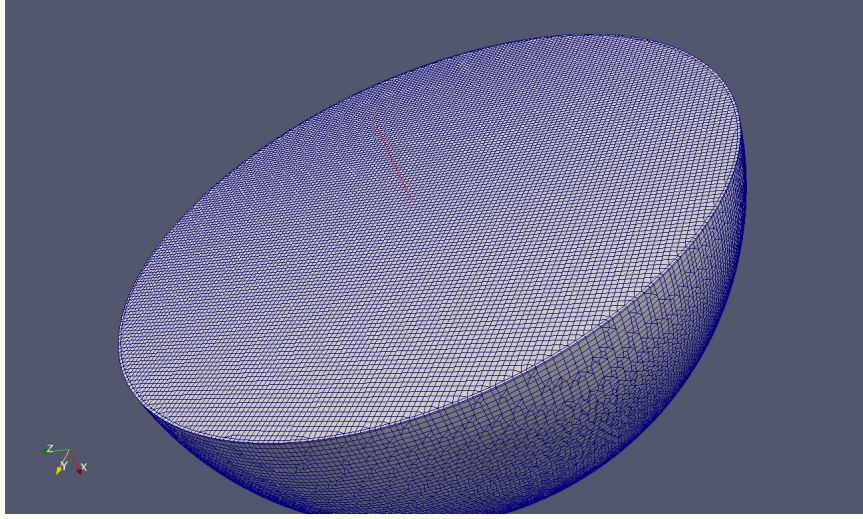


Figure 4. A mesh for the ball obtained from a cubic uniformly graded mesh $128 \times 128 \times 128$ after applying the OpenFOAM tool `snappyHexMesh`. The ball is cut by a plane, to show interior of the mesh.

5.

6.4 Self-focusing case

Firstly, equation (3) is solved for the case of $C_1 = 1$ and $C_2 = 0$. The positive non-linearity causes self-focusing effects. Finally, parts of the wave function collapse, see Figure 6.

Similarly, equation (3) is solved for the case of $C_1 = 0$ and $C_2 = -1$. Again, a positive non-linearity causes self-focusing effects with final collapse, see Figure 7.

To avoid the collapse, we tried to solve equation (3) with a coefficient in front of Laplacian having a negative imaginary part, $C_0 = 1 - i$, that causes some damping, and with a positive cubic term, $C_1 = 1$. The result is shown on Figure 8. Here, the function is gradually focusing into a narrow region, but singularities don't develop, and the collapse does not occur.

6.5 Defocusing case

Defocusing occurs when the non-linearity is negative. Figure 9 shows the case of $C_1 = -1$ and $C_2 = 0$, and Figure 10 the case of $C_1 = 0$ and $C_2 = -1$. De-focusing causes the wave function to expand explosively, and finally disperse over the volume.

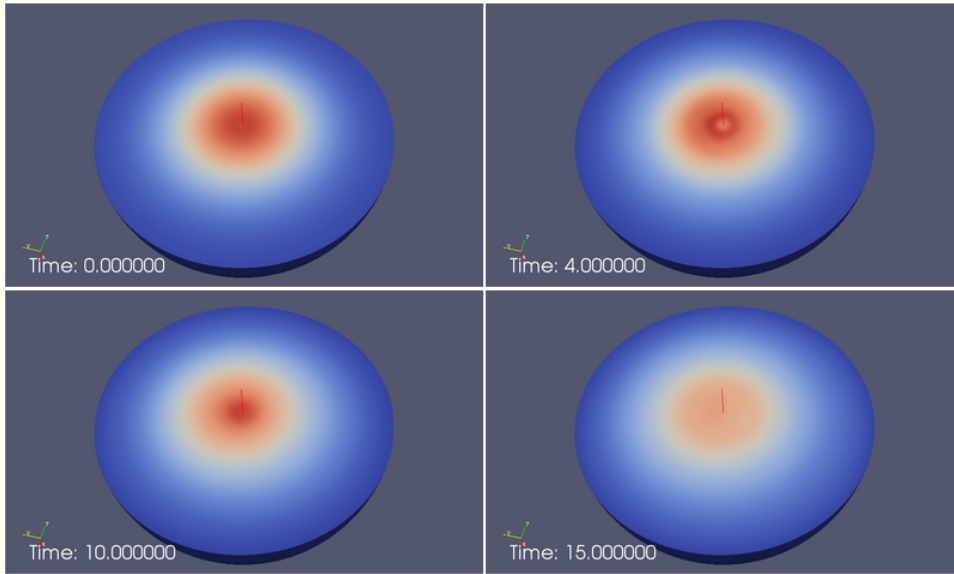


Figure 5. Solution of linear Schrödinger equation $i\frac{\partial\Psi}{\partial t} + \Delta\Psi = 0$. Density plot shows absolute value of the function. Blue color corresponds to $|\Psi| = 0$, and red color to maximal possible absolute value at $t = 0$. Wave packet slowly disperses with time.

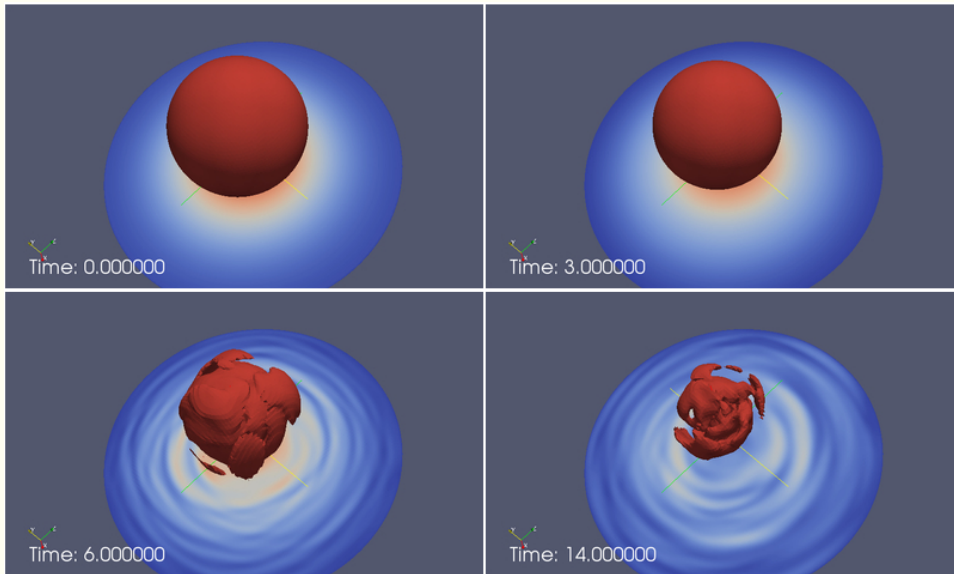


Figure 6. Solution of non-linear Schrödinger equation $i\frac{\partial\Psi}{\partial t} + \Delta\Psi + |\Psi|^2\Psi = 0$. There are two plots combined together, a contour plot of $|\Psi|$ (a ball), and a density plot on the sphere sliced at $x = 8$. Wave packet shrinks with time and finally collapses.

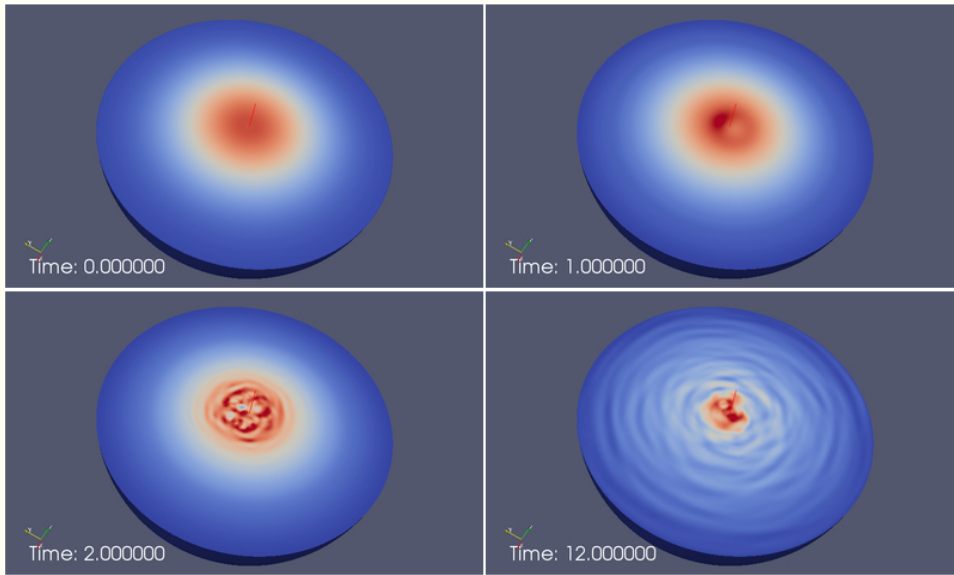


Figure 7. Solution of non-linear Schrödinger equation $i\frac{\partial\Psi}{\partial t} + \Delta\Psi + |\Psi|^4\Psi = 0$. Wave packet shrinks with time and finally collapses.

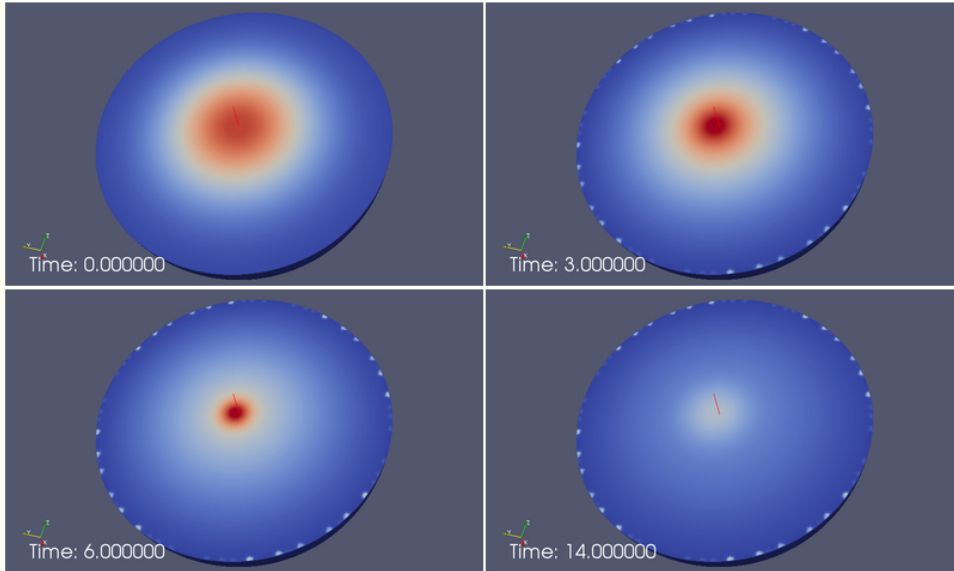


Figure 8. Solution of non-linear Schrödinger equation with a complex-valued factor in front of Laplacian, $i\frac{\partial\Psi}{\partial t} + (1-i)\Delta\Psi + |\Psi|^2\Psi = 0$ corresponding to a decaying state. Wave packet shrinks with time, focuses into one spot and finally decays, but without a collapse. White spots at the edges reflect numerical instabilities caused by large Courant number when the grid spacing becomes relatively small.

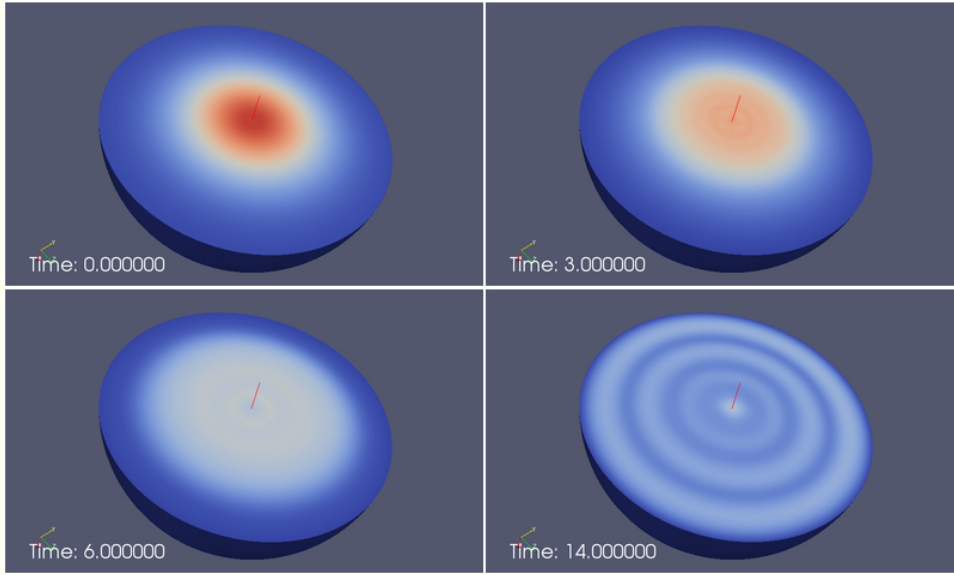


Figure 9. Solution of non-linear Schrödinger equation $i\frac{\partial\Psi}{\partial t} + \Delta\Psi - |\Psi|^2\Psi = 0$. Wave packet blows up and finally disperses over the volume of the ball.

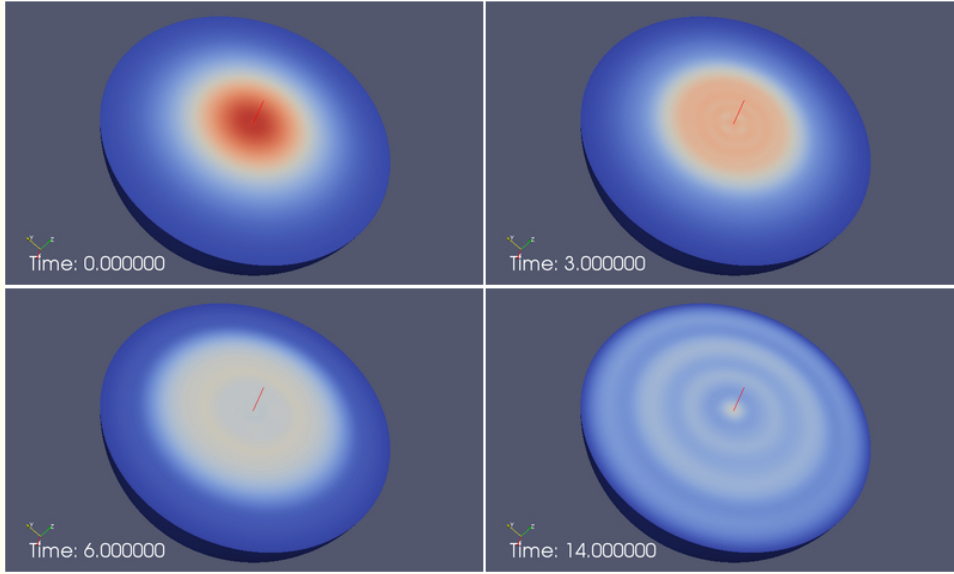


Figure 10. Solution of non-linear Schrödinger equation $i\frac{\partial\Psi}{\partial t} + \Delta\Psi - |\Psi|^4\Psi = 0$. Negative non-linearity causes the wave packet to blow up. Finally, it disperses over the volume of the ball.

6.6 Stable case

Stabilization occurs when cubic and quintic non-linearities have opposite sign. Figure 11 shows the case of $C_1 = C_2 = 1$. The wave packet becomes more compact with time, and finally stabilizes.

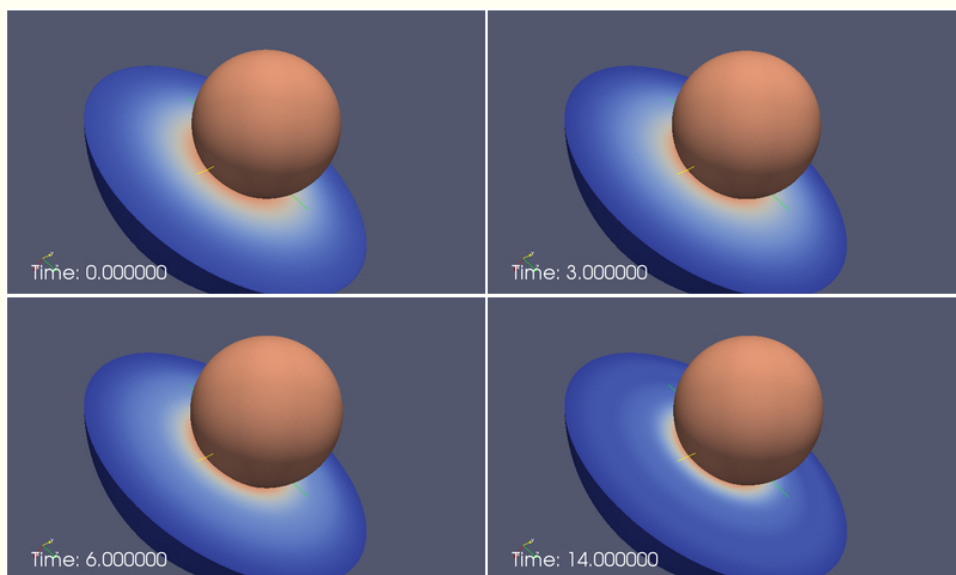


Figure 11. Solution of non-linear Schrödinger equation with combined cubic-quintic nonlinearities of opposite signs, $i\frac{\partial\Psi}{\partial t} + \Delta\Psi + |\Psi|^2\Psi - |\Psi|^4\Psi = 0$ corresponding to a stable soliton. Wave packet originally excretes small fractions of the wave function, and finally becomes stable. There are two plots combined together, a contour plot of $|\Psi|$ (a ball), and a density plot on the sphere sliced at $x = 8$.

7 Stability of cylindrical solitons

Here, we consider initial condition

$$\Psi_{(t=0)} = Ae^{-\frac{x^2+y^2}{2R^2}} (1 + 0.1 \cos(5\pi z/L)), \quad (7)$$

$$A = 0.8, \quad R = 3.8. \quad (8)$$

We also add an extra term in the nonlinear equation, $-C_2|\Psi|^4\Psi$. The rectangular parallelepiped is square-shaped in (x, y) -plane with size of square $a = 32$, and has length $L = 48$ in z -direction. The results are shown on Fig. 12.

The results with larger size of the region, $a = 48$ and $L = 72$ (not sure about exact value) are shown on Fig. 13.

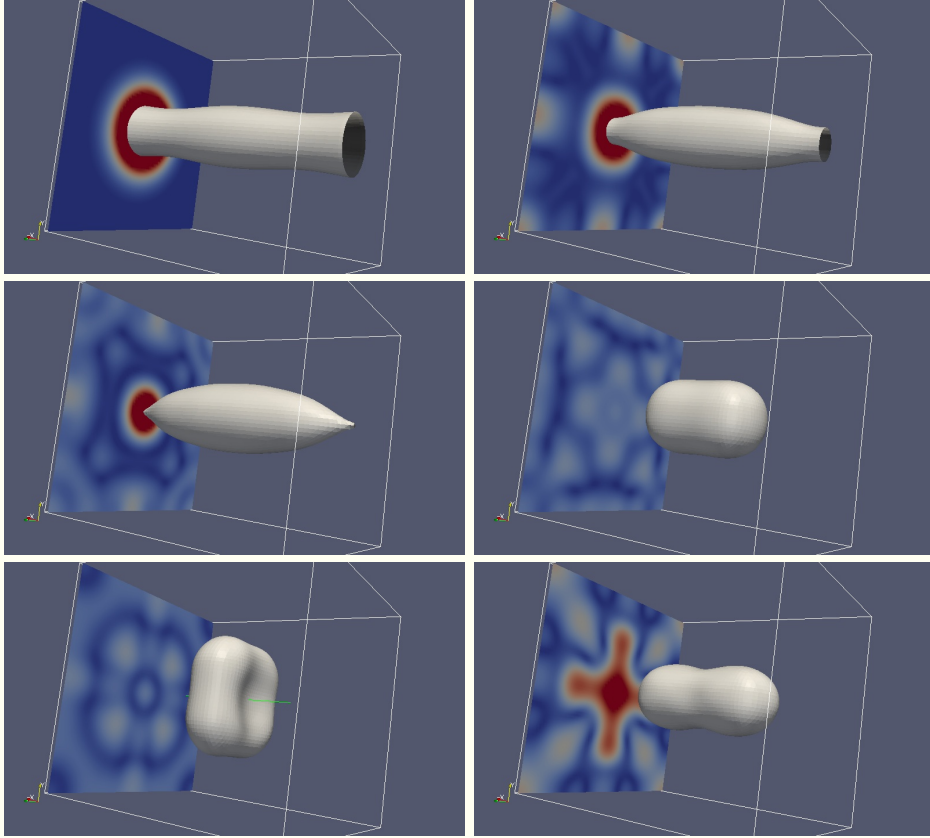


Figure 12. Instability of a cylindrical solution of non-linear Schrödinger equation $i\frac{\partial\Psi}{\partial t} + \Delta\Psi + |\Psi|^2\Psi - |\Psi|^4\Psi = 0$ with a cylindrical-symmetry initial condition given by equation (7). The panels show calculated absolute value of the wave function in equation (1) at $t = 0, 256 \times 0.3, 296 \times 0.3, 352 \times 0.3, 453 \times 0.3$, and 664×0.3 .

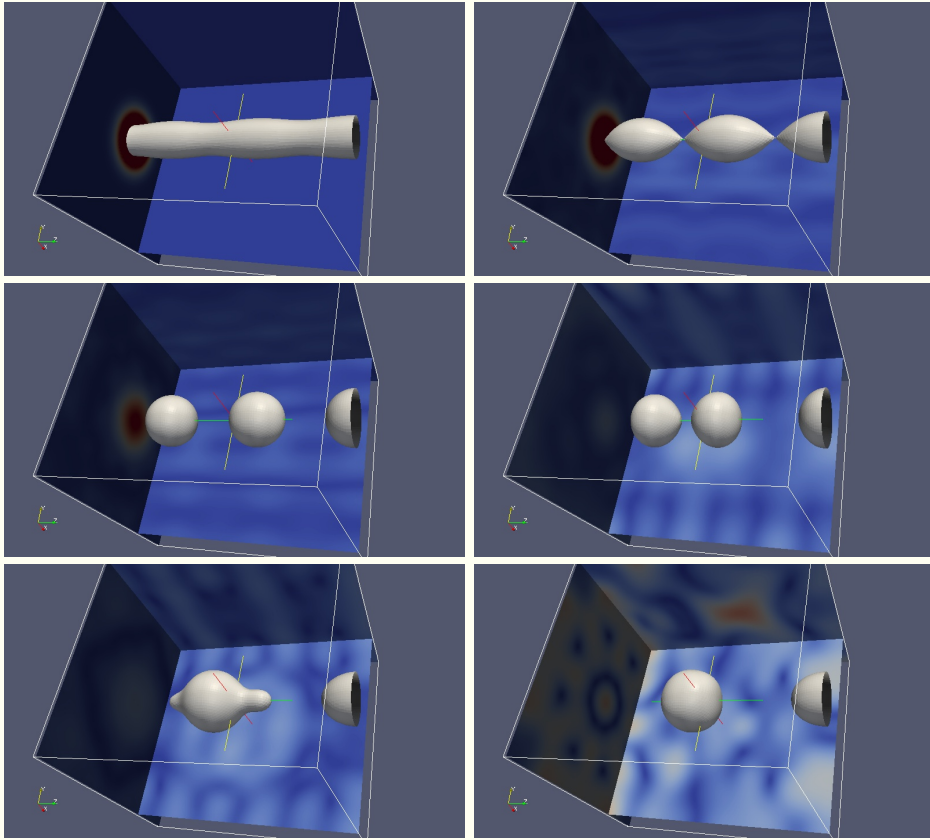


Figure 13. Instability of a cylindrical solution of non-linear Schrödinger equation $i\frac{\partial\Psi}{\partial t} + \Delta\Psi + |\Psi|^2\Psi - |\Psi|^4\Psi = 0$ with a cylindrical-symmetry initial condition given by equation (7). The panels show calculated absolute value of the wave function in equation (1) at $t = 0$, 66×0.3 , 83×0.3 , 534×0.3 , 571×0.3 , and 812×0.3 .

8 Ginzburg–Landau equation

The equation to solve is [communications with Najdan]

$$i \frac{\partial \Psi}{\partial t} + (1 - i\beta) \Delta \Psi - i\delta \Psi + (1 - i\epsilon) |\Psi|^2 \Psi - (1 + i\mu) |\Psi|^4 \Psi = 0, \quad (9)$$

with Neumann (zero gradient) boundary conditions. The constants in equation (9) are as follows,

$$\beta = 0.05, \quad \delta = -0.01, \quad \epsilon = 0.45, \quad \mu = -0.8. \quad (10)$$

The equation is solved in two dimensions inside a square of size 96 with the center at the origin. The mesh is $256 \times 256 \times 1$.

The initial condition is given as

$$\Psi_{(t=0)} = A(x, y) A_0 e^{-\frac{(x-x_0)^2 + (y-y_0)^2}{2R^2}}, \quad (11)$$

$$A_0 = 0.87, \quad x_0 = 4, \quad y_0 = 2, \quad R = 4.03, \quad (12)$$

and

$$A(x, y) = 1 + 0.1 \frac{x + iy}{(x^2 + y^2)^{1/2}} \quad (13)$$

where the second term is added in order to break rotation symmetry.

The time step is chosen as $\Delta t = 0.01$ that is around 50% of maximal possible time step for which Courant number allows convergence of the method. The function was computed up to $t_{\max} = 1000$.

Results of calculation are shown on Figure 14. With passing of time, the soliton becomes elongated and starts rotating. The radial symmetry of the function breaks at around time $t = 500$.

References

- [1] V. Skarka, D. V. Timotijević, and N. B. Aleksić. Journal of Optics A: Pure and Applied Optics, vol. 10, 075102, 2008.
- [2] N. B. Aleksić, V. Skarka, D. V. Timotijević, and D. Gauthier. Phys. Rev. A, vol. 75, 061802(R), 2007.

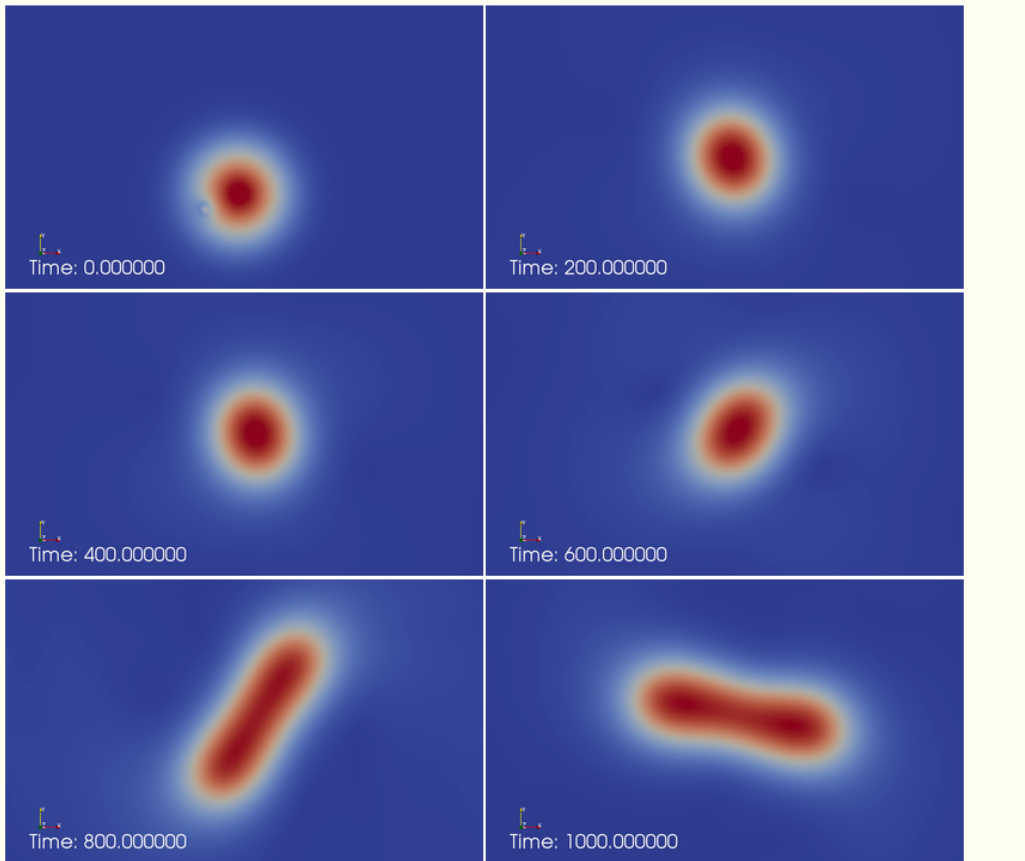


Figure 14. Solution of Ginzburg–Landau equation given by equation (11) with initial condition given by equation (13) At small times, the solution is a Gaussian-like, with radial symmetry. Then, soliton elongates in one direction and starts rotating.